

Modul 6: Pemrograman Berorientasi Objek (OOP) pada PHP

6.1 Tujuan

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

1. Memahami konsep dasar Pemrograman Berorientasi Objek (OOP) dalam PHP.
2. Menggunakan tipe data object dalam PHP.
3. Membuat dan menggunakan class serta object.
4. Memahami konsep constructor dan destructor.
5. Menerapkan konsep inheritance dalam OOP PHP.

6.2 Alat dan Bahan

Untuk melaksanakan praktikum ini, diperlukan:

1. Komputer dengan sistem operasi Windows, macOS, atau Linux.
2. Web server seperti **XAMPP**.
3. Editor teks atau IDE seperti **VS Code**, **Sublime Text**, atau **PHPStorm**.
4. Browser web (Google Chrome, Mozilla Firefox, atau lainnya).

6.3 Tugas Pendahuluan

Sebelum memulai praktikum, lakukan tugas berikut:

1. Apa yang dimaksud dengan OOP dalam PHP?
2. Sebutkan dan jelaskan manfaat menggunakan OOP dalam pengembangan perangkat lunak?
3. Apa perbedaan antara class dan object dalam PHP?
4. Jelaskan fungsi dari constructor dan destructor dalam PHP?
5. Apa yang dimaksud dengan inheritance dalam OOP? Berikan contoh penerapannya dalam PHP?

6.4 Materi Praktikum

6.4.1 Pengertian OOP

Pemrograman Berorientasi Objek (OOP) adalah metode pemrograman yang berfokus pada pembuatan objek yang berisi data dan fungsi. Berbeda dengan **pemrograman prosedural** yang menggunakan fungsi untuk memproses data, OOP menggabungkan data dan fungsinya dalam satu kesatuan, yaitu **object**.

OOP memiliki beberapa **keunggulan** dibandingkan dengan **pemrograman prosedural**:

- Eksekusi kode lebih cepat dan efisien.
- Struktur kode lebih jelas dan mudah dikelola.
- Menghindari pengulangan kode dengan menerapkan prinsip DRY (*Don't Repeat Yourself*), sehingga lebih mudah untuk dipelihara, dimodifikasi, dan diperbaiki.
- Memungkinkan pembuatan aplikasi yang dapat digunakan kembali dengan lebih sedikit kode dan waktu pengembangan yang lebih singkat.

Prinsip DRY bertujuan untuk mengurangi duplikasi kode dengan menempatkan kode yang sering digunakan di satu tempat agar dapat digunakan kembali tanpa perlu menuliskannya berulang kali.

6.4.2 Pengertian dan Cara Penulisan Class dalam Pemrograman Berorientasi Objek (OOP)

Dalam pemrograman berorientasi objek (OOP), **class** berfungsi sebagai kerangka kerja atau template yang digunakan untuk membuat objek. **Class** dapat dianalogikan sebagai cetakan yang menentukan bagaimana suatu objek akan dibuat, termasuk atribut (**property**) dan perilaku (**method**) yang dimilikinya. Dengan adanya **class**, pengembang dapat mengorganisir kode dengan lebih terstruktur, memudahkan pemeliharaan, dan meningkatkan efisiensi dalam pengembangan perangkat lunak.

Berikut adalah contoh penulisan class OOP PHP:

```
<?php
class Kendaraan {
    public $merk;
    public $warna;

    public function tampilkanInfo() {
        return "Kendaraan ini adalah $this->merk dengan warna $this->warna.";
    }
}
?>
```

Class di atas memiliki dua **properti** (merk dan warna) serta satu **method** (tampilkanInfo) yang mengembalikan informasi tentang kendaraan tersebut.

6.4.3 Pengertian dan Cara Penulisan Properti dalam Pemrograman Berorientasi Objek (OOP)

Dalam pemrograman berorientasi objek (OOP) pada PHP, **properti** adalah data atau atribut yang terdapat dalam sebuah **class**. Properti digunakan untuk menyimpan informasi terkait dengan suatu objek. Misalnya, dalam **class Mobil**, properti yang dapat dimiliki adalah **warna**, **ukuran**, dan **merek**. Sedangkan dalam **class Manusia**, properti yang bisa digunakan antara lain **nama**, **warna rambut**, dan **warna kulit**.

Secara fungsional, **properti** dalam class memiliki kegunaan yang sama seperti variabel pada PHP, yaitu untuk menyimpan nilai yang dapat digunakan dalam berbagai proses di dalam program.

Berikut adalah contoh penulisan properti di dalam class OOP PHP:

```

<?php
class Mobil {
    public $warna;      // Properti warna
    private $merek;     // Properti merek (hanya bisa diakses dalam class)
    protected $ukuran; // Properti ukuran (bisa diakses oleh class turunan)

    // Konstruktor untuk menginisialisasi properti
    public function __construct($warna, $merek, $ukuran) {
        $this->warna = $warna;
        $this->merek = $merek;
        $this->ukuran = $ukuran;
    }

    // Method untuk menampilkan properti
    public function getMerek() {
        return $this->merek;
    }
}

// Membuat objek dari class Mobil
$mobilBaru = new Mobil("Merah", "Toyota", "Sedang");

// Mengakses properti publik
echo "Warna mobil: " . $mobilBaru->warna;
?>

```

Dalam contoh di atas:

- **public** berarti properti bisa diakses dari luar class.
- **private** berarti properti hanya bisa diakses dari dalam class itu sendiri.
- **protected** berarti properti bisa diakses oleh class tersebut dan class turunannya.

6.4.4 Pengertian dan Cara Penulisan Method dalam Pemrograman Berorientasi Objek (OOP)

Dalam pemrograman berorientasi objek (OOP) di PHP, **method** merupakan sebuah fungsi yang terdapat di dalam **class** dan digunakan untuk menjalankan aksi tertentu. Method dapat diibaratkan sebagai perilaku atau tindakan yang bisa dilakukan oleh suatu objek.

Sebagai contoh, pada **class** bernama Mobil, method yang bisa dimiliki antara lain **maju()**, **berhenti()**, dan **belok()**, yang masing-masing merepresentasikan tindakan yang dapat dilakukan oleh objek dari **class** tersebut.

Method dalam **class** OOP PHP dituliskan dengan menggunakan kata kunci **function**, diikuti dengan nama method tersebut. Method juga dapat memiliki parameter untuk menerima input serta dapat mengembalikan nilai sebagai hasil eksekusinya.

Berikut adalah contoh penulisan Method di dalam class OOP PHP:

```

<?php
class Mobil {
    public function maju() {
        echo "Mobil bergerak maju";
    }

    public function berhenti() {
        echo "Mobil berhenti";
    }

    public function belok($arah) {
        echo "Mobil berbelok ke " . $arah;
    }
}

// Membuat objek dari class Mobil
$myCar = new Mobil();

// Memanggil method
$myCar->maju();
$myCar->berhenti();
$myCar->belok("kanan");
?>

```

Dalam contoh di atas, terdapat tiga method dalam **class** Mobil, yaitu **maju()**, **berhenti()**, dan **belok(\$arah)**. Method **belok(\$arah)** menerima parameter **\$arah**, yang menentukan ke arah mana mobil akan berbelok.

6.4.5 Pengertian dan Cara Penulisan Method dalam Pemrograman Berorientasi Objek (OOP)

Dalam pemrograman berorientasi objek (OOP), **object** adalah hasil instansiasi dari sebuah **class**. Object memungkinkan kita untuk mengakses dan mengelola data serta fungsi yang ada dalam class. Dengan kata lain, object adalah representasi nyata dari sebuah class yang dapat digunakan dalam program.

Berikut adalah contoh penulisan Object di dalam class OOP PHP:

```

<?php
// Mendefinisikan class Mobil
class Mobil {
    public $warna;

    // Method untuk menetapkan warna mobil
    public function setWarna($warna) {
        $this->warna = $warna;
    }

    // Method untuk mendapatkan warna mobil
    public function getWarna() {
        return $this->warna;
    }
}

// Membuat object dari class Mobil
$mobil1 = new Mobil();
$mobil1->setWarna("Merah");

$mobil2 = new Mobil();
$mobil2->setWarna("Biru");

// Menampilkan warna mobil
echo "Warna mobil pertama: " . $mobil1->getWarna() . "<br>";
echo "Warna mobil kedua: " . $mobil2->getWarna();
?>

```

Pada contoh di atas, **\$mobil1** dan **\$mobil2** merupakan object dari class **Mobil**. Setiap object memiliki warna yang berbeda sesuai dengan nilai yang diberikan.

6.4.6 Pengertian Konstruktor dan Destruktor dalam OOP PHP

Dalam pemrograman berorientasi objek (OOP) di PHP, **konstruktor** dan **destruktor** adalah metode khusus yang digunakan untuk mengelola siklus hidup objek.

- **Konstruktor (__construct)** adalah metode yang otomatis dijalankan saat sebuah objek dari kelas dibuat. Biasanya, metode ini digunakan untuk menginisialisasi nilai awal atau melakukan tugas persiapan lain yang diperlukan oleh objek.
- **Destruktor (__destruct)** adalah metode yang dijalankan secara otomatis saat sebuah objek tidak lagi digunakan atau dihapus. Metode ini berguna untuk membersihkan sumber daya yang telah digunakan oleh objek, seperti menutup koneksi database atau menghapus file sementara.

Berikut adalah contoh penulisan Konstruktor dan Destruktor dalam OOP PHP:

```
<?php
class Mobil {
    public $merk;
    public $warna;

    // Konstruktor: Dijalankan saat objek dibuat
    public function __construct($merk, $warna) {
        $this->merk = $merk;
        $this->warna = $warna;
        echo "Mobil $this->merk dengan warna $this->warna telah dibuat.<br>";
    }

    public function jalankan() {
        echo "Mobil $this->merk sedang berjalan.<br>";
    }

    // Destruktor: Dijalankan saat objek tidak lagi digunakan
    public function __destruct() {
        echo "Mobil $this->merk telah dihentikan dan dihapus dari memori.<br>";
    }
}

// Membuat objek Mobil
$mobil1 = new Mobil("Toyota", "Merah");
$mobil1->jalankan();

// Objek akan dihapus secara otomatis setelah skrip selesai
?>
```

- **Saat objek Mobil dibuat**, konstruktor (`__construct`) akan otomatis dijalankan dan menampilkan pesan bahwa mobil telah dibuat.
- **Metode jalankan()** dipanggil untuk mensimulasikan mobil yang sedang berjalan.
- **Ketika skrip selesai atau objek dihapus**, destruktur (`__destruct`) akan dijalankan secara otomatis dan menampilkan pesan bahwa mobil telah dihentikan.

6.4.7 Pengertian Enkapsulasi (Encapsulation) dalam OOP PHP

Enkapsulasi adalah konsep dalam pemrograman berorientasi objek (OOP) yang digunakan untuk mengatur struktur dalam sebuah **class** dengan cara mengontrol hak akses terhadap **property** (atribut) dan **method** (fungsi). Tujuan utama dari enkapsulasi adalah untuk **menyembunyikan informasi** dan membatasi akses langsung ke data demi alasan keamanan dan kestabilan sistem.

Dalam OOP, terdapat tiga tingkatan hak akses yang digunakan untuk mengontrol akses ke atribut dan metode dalam sebuah **class**, yaitu:

1. **Public** – Dapat diakses dari mana saja, baik di dalam maupun di luar kelas.
2. **Private** – Hanya dapat diakses di dalam kelas itu sendiri.
3. **Protected** – Dapat diakses oleh kelas itu sendiri dan kelas turunannya (inheritance).

Berikut adalah contoh penulisan Enkapsulasi (Encapsulation) dalam OOP PHP:

```
<?php
class Mobil {
    public $merk; // Bisa diakses dari mana saja
    private $kecepatanMaksimum; // Hanya bisa diakses dalam kelas ini
    protected $jumlahPintu; // Bisa diakses dalam kelas ini dan kelas turunannya

    public function __construct($merk, $kecepatan, $pintu) {
        $this->merk = $merk;
        $this->kecepatanMaksimum = $kecepatan;
        $this->jumlahPintu = $pintu;
    }

    public function getKecepatanMaksimum() {
        return $this->kecepatanMaksimum; // Mengakses properti private melalui metode dalam kelas
    }

    protected function getJumlahPintu() {
        return $this->jumlahPintu; // Mengakses properti protected
    }
}

// Membuat objek Mobil
$mobil1 = new Mobil("Toyota", 180, 4);
echo "Merk Mobil: " . $mobil1->merk . "<br>"; // Bisa diakses karena public
echo "Kecepatan Maksimum: " . $mobil1->getKecepatanMaksimum() . " km/jam<br>";
// Menggunakan method public untuk mengakses properti private

// $mobil1->kecepatanMaksimum; // ERROR: Tidak bisa diakses langsung karena private
// $mobil1->jumlahPintu; // ERROR: Tidak bisa diakses langsung karena protected
?>
```

- **Properti \$merk (Public)** Dapat diakses dari luar kelas, sehingga bisa langsung diubah atau dibaca.
- **Properti \$kecepatanMaksimum (Private)** Hanya bisa diakses dari dalam kelas, sehingga perlu menggunakan metode getKecepatanMaksimum() untuk mendapatkan nilainya.
- **Properti \$jumlahPintu (Protected)** Tidak bisa diakses langsung dari luar kelas, tetapi bisa digunakan oleh kelas turunan.

6.4.8 Pengertian Inheritance (Pewarisan Sifat) dalam OOP PHP

Inheritance atau **pewarisan sifat** dalam OOP (Pemrograman Berorientasi Objek) adalah konsep di mana sebuah kelas baru dapat mewarisi sifat atau karakteristik dari kelas lain. Dengan kata lain, kita dapat membuat kelas baru yang terhubung dengan kelas lain untuk mengambil informasi atau data dari kelas tersebut. Kelas baru ini disebut sebagai **subclass** atau **child class**, sementara kelas yang menjadi sumber sifat disebut **parent class**.

Berikut adalah contoh penulisan Inheritance dalam OOP PHP:

```
<?php
class Mobil { // Kelas induk (Parent Class)
    public $merk; // Properti
    public $warna;
    public function __construct($merk, $warna) { // Konstruktor
        $this->merk = $merk;
        $this->warna = $warna;
    }
    public function tampilkanInfo() { // Metode
        return "Mobil Merk: $this->merk, Warna: $this->warna";
    }
}

class MobilSport extends Mobil { // Kelas turunan (Child Class)
    public $kecepatanMaks; // Menambahkan properti khusus
    public function __construct($merk, $warna, $kecepatanMaks) { // Konstruktor
        parent::__construct($merk, $warna); // Memanggil konstruktor kelas induk
        $this->kecepatanMaks = $kecepatanMaks;
    }
    public function tampilkanInfoSport() { // Menambahkan metode khusus
        return $this->tampilkanInfo() . ", Kecepatan Maksimal: $this->kecepatanMaks km/jam";
    }
}

$mobil = new MobilSport("Ferrari", "Merah", 350); // Membuat objek dari kelas MobilSport
echo $mobil->tampilkanInfoSport(); // Menampilkan informasi mobil sport
?>
```

- **Mobil** adalah kelas induk (parent class) yang memiliki properti dan metode dasar seperti merk, warna, dan metode untuk menampilkan informasi.
- **MobilSport** adalah kelas turunan (child class) yang mewarisi sifat dari kelas **Mobil** dan menambahkan properti serta metode baru, seperti kecepatan maksimal.

6.5 Format laporan

Silakan kerjakan tugas praktikum berikut ini dan kumpulkan dalam bentuk laporan dengan format sebagai berikut:

1. Halaman Sampul (Cover)
 - Judul Praktikum
 - Nama Mahasiswa
 - NIM
 - Kelas
 - Mata Kuliah
2. Tujuan Praktikum
3. Alat dan Bahan
4. Langkah-Langkah Praktikum
5. Hasil dan Dokumentasi Program

6. Analisis dan Pembahasan
7. Kesimpulan
8. Lampiran berisi Link repositori GitHub
9. Presentasikan hasil praktikumnya!

6.6 Tugas Praktikum

1. Buatlah aplikasi web sederhana berbasis *database* yang mengimplementasikan fitur CRUD (*Create, Read, Update, Delete*) serta *upload file* sesuai rancangan Tugas Besar Saudara. Aplikasi minimal memiliki satu halaman utama (*dashboard*) yang hanya dapat diakses dan ditampilkan setelah login berhasil. Pada proses penyimpanan data, gunakan satu tombol “Simpan” yang berfungsi untuk menjalankan proses CRUD sekaligus mengunggah *file* dalam satu kali submit. Seluruh program wajib menerapkan prinsip OOP (*Object-Oriented Programming*) dalam perancangan kode.